



Computational Physics @ NuSTEAM

Lecture 1

- *Introduction*
- *Data visualization (plotting)*
- *Floating-point precision*

June 11, 2025

Instructor: Volodymyr Vovchenko (vvovchenko@uh.edu)

based on Computations Physics course at UH

GitHub: <https://github.com/vlvovch/PHYS6350-ComputationalPhysics/tree/nusteam>

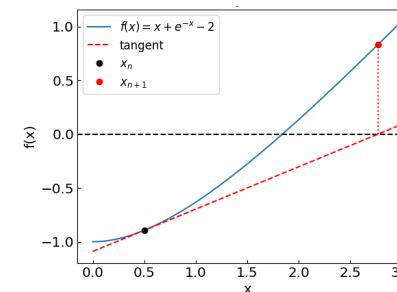
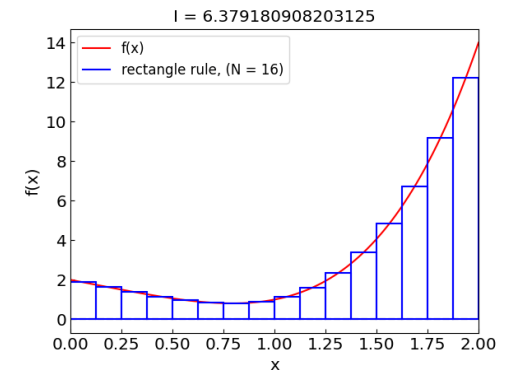
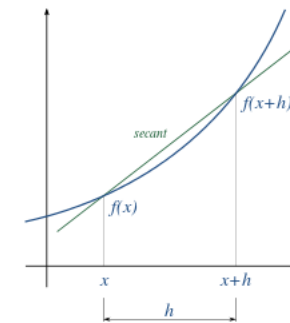
Online textbook: <https://vovchenko.net/computational-physics>

Computational Physics

Description: Application of numerical methods to solve problems in physics, which lies at the intersection of physics, math, and computer science.

Common applications:

- Numerical calculus
 - Numerical integration
 - Numerical derivatives
- Numerical algebra
 - Systems of linear and non-linear equations
 - Matrix methods
- Ordinary differential equations
 - Classical mechanics problems
- Partial differential equations
 - Classical and quantum mechanics problems
- Random numbers and Monte Carlo methods



$$\frac{dx}{dt} = f(x, t),$$

$$\frac{\partial \psi}{\partial t} \approx \frac{i\hbar}{2ma^2} [\psi(x+a, t) - 2\psi(x, t) + \psi(x-a, t)].$$

Data visualization

- Line plots
- Scatter plots
- Contour/density plots (2D data)

References: [Matplotlib documentation](#)

Jupyter Notebook: 1_Plotting-NuSTEAM2025.ipynb

Plotting the data

Computer programs produce numerical data

Numbers alone do not always make it easy to understand the structure of behavior of the system

Consider a function $y = \sin(x)$

Let us calculate it for 10 equidistant points in the interval $x = 0 \dots 10$

Plotting the data

Computer programs produce numerical data

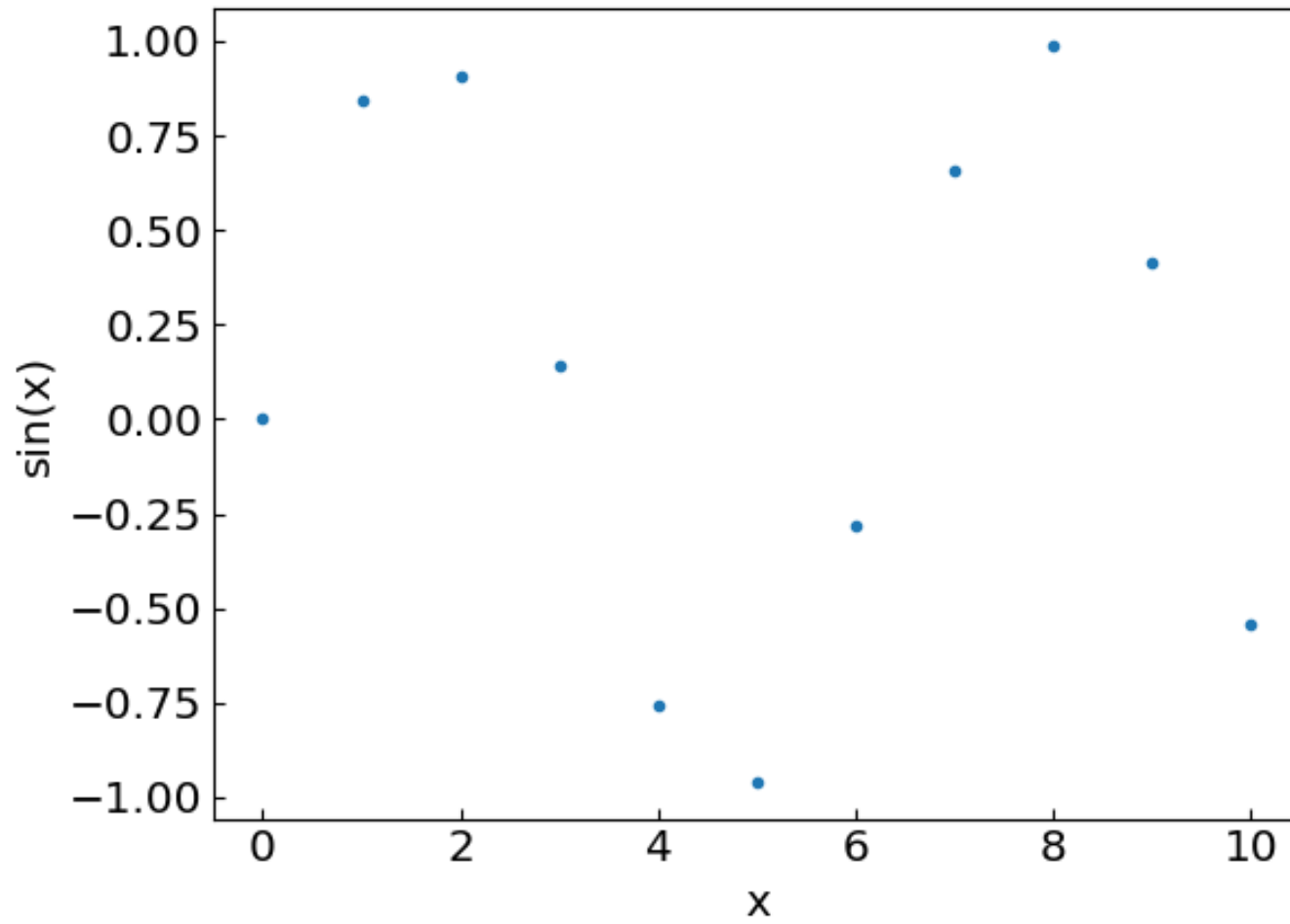
Numbers alone do not always make it easy to understand the structure of behavior of the system

Consider a function $y = \sin(x)$

Let us calculate it for 10 equidistant points in the interval $x = 0 \dots 10$

x	sin(x)
0	0.
1	0.841471
2	0.9092974
3	0.14112
4	-0.7568025
5	-0.9589243
6	-0.2794155
7	0.6569866
8	0.9893582
9	0.4121185
10	-0.5440211

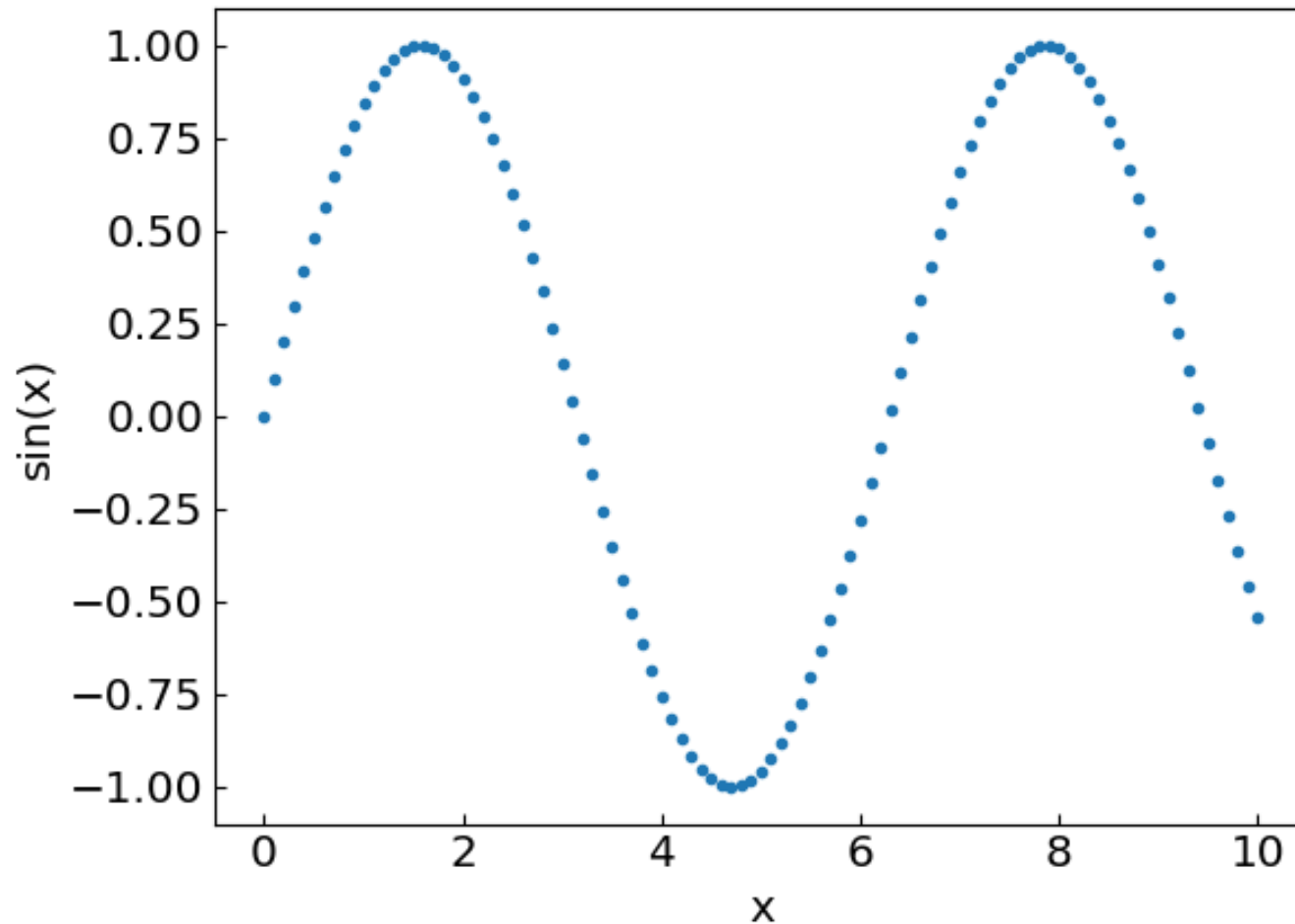
Putting it on a graph



x	sin(x)
0	0.
1	0.841471
2	0.9092974
3	0.14112
4	-0.7568025
5	-0.9589243
6	-0.2794155
7	0.6569866
8	0.9893582
9	0.4121185
10	-0.5440211

Let us add more points...

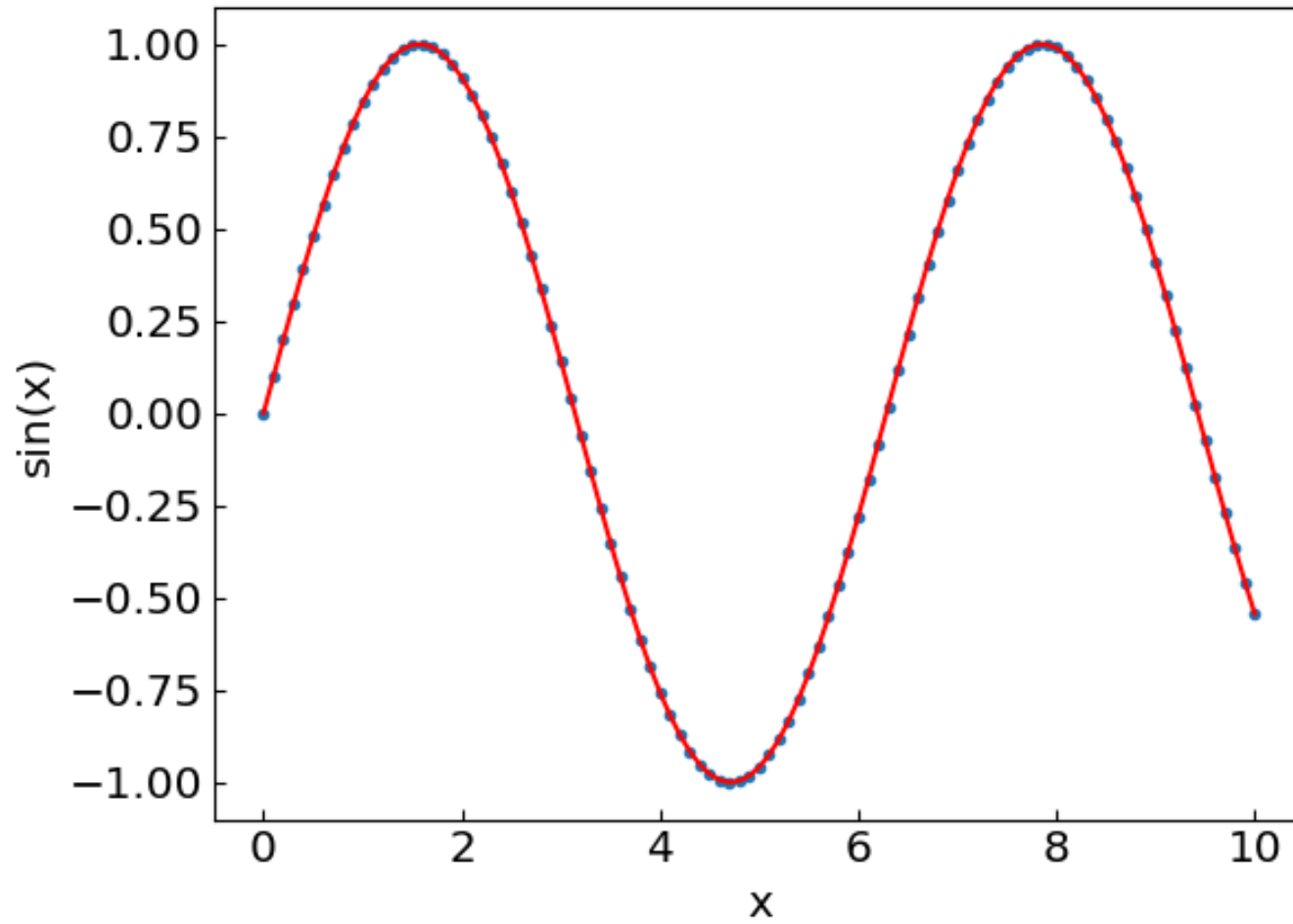
Putting it on a graph



x	sin(x)
0.	0.
0.1	0.09983342
0.2	0.1986693
0.3	0.2955202
0.4	0.3894183
0.5	0.4794255
0.6	0.5646425
0.7	0.6442177
0.8	0.7173561
0.9	0.7833269
1.	0.841471
	⋮
9.9	-0.4575359
10.	-0.5440211

Now we have enough points to join them by a smooth line

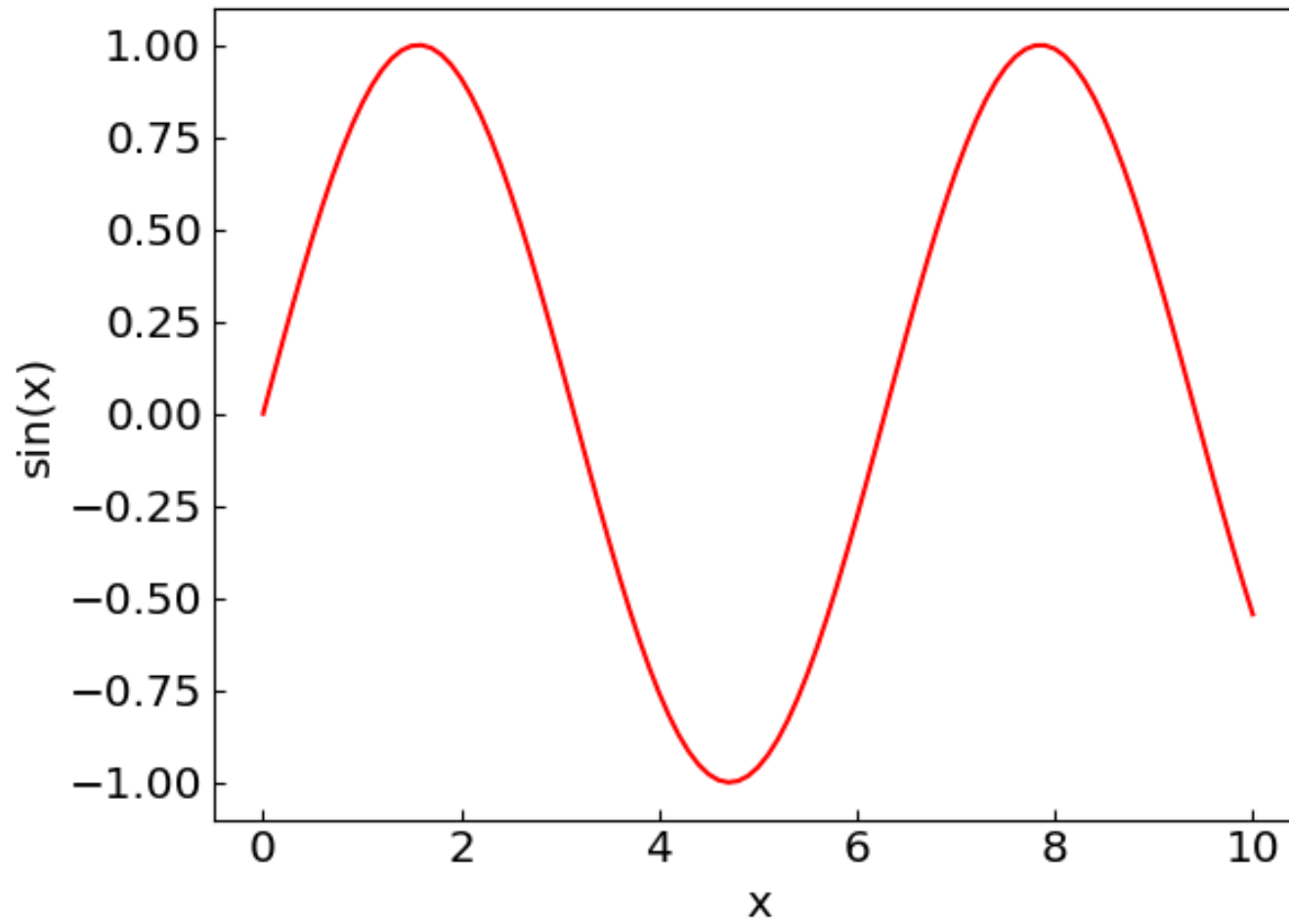
Putting it on a graph



x	sin(x)
0.	0.
0.1	0.09983342
0.2	0.1986693
0.3	0.2955202
0.4	0.3894183
0.5	0.4794255
0.6	0.5646425
0.7	0.6442177
0.8	0.7173561
0.9	0.7833269
1.	0.841471
	⋮
9.9	-0.4575359
10.	-0.5440211

Now we have enough points to join them by a smooth line

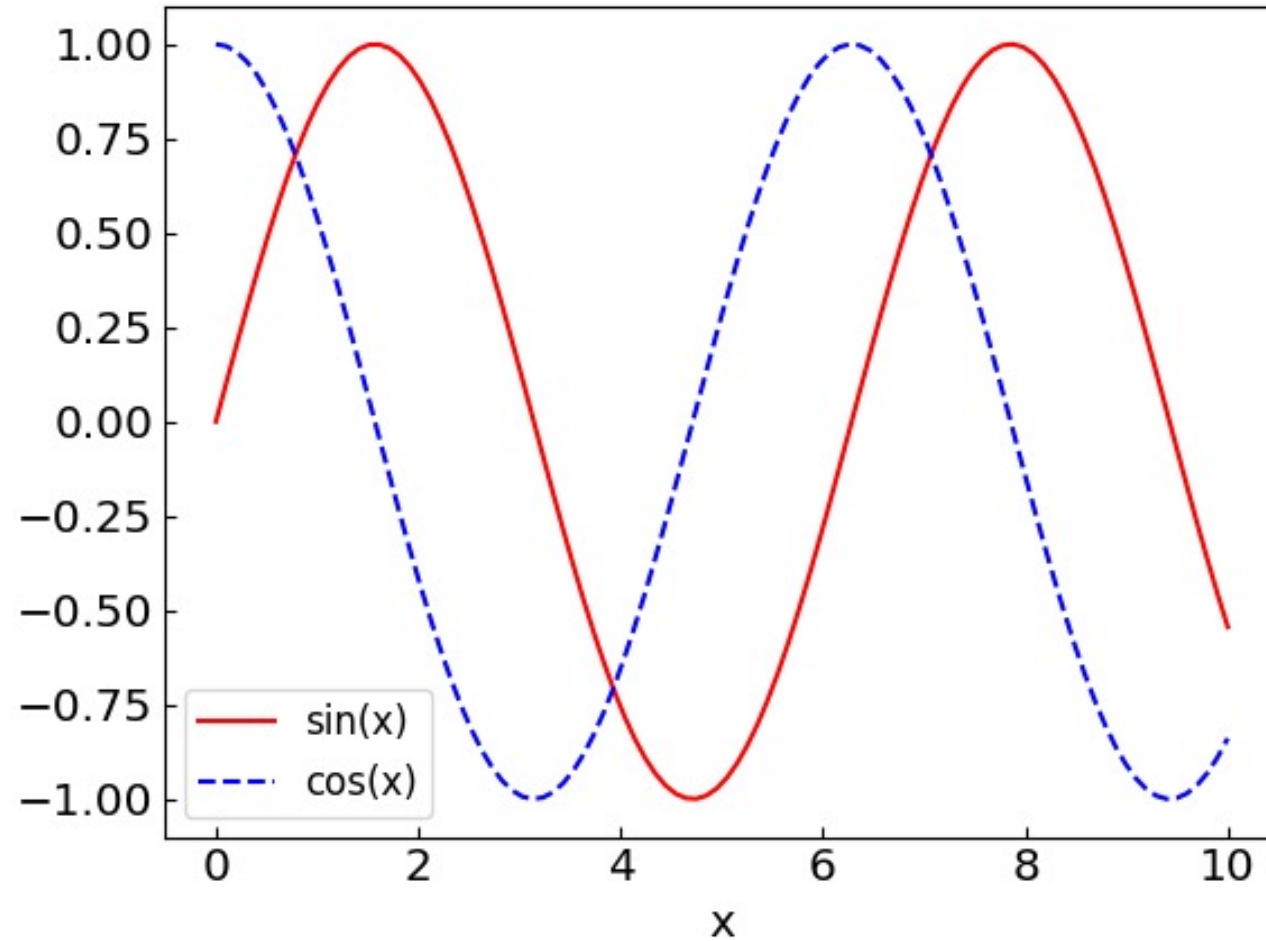
Putting it on a graph



x	sin(x)
0.	0.
0.1	0.09983342
0.2	0.1986693
0.3	0.2955202
0.4	0.3894183
0.5	0.4794255
0.6	0.5646425
0.7	0.6442177
0.8	0.7173561
0.9	0.7833269
1.	0.841471
	⋮
9.9	-0.4575359
10.	-0.5440211

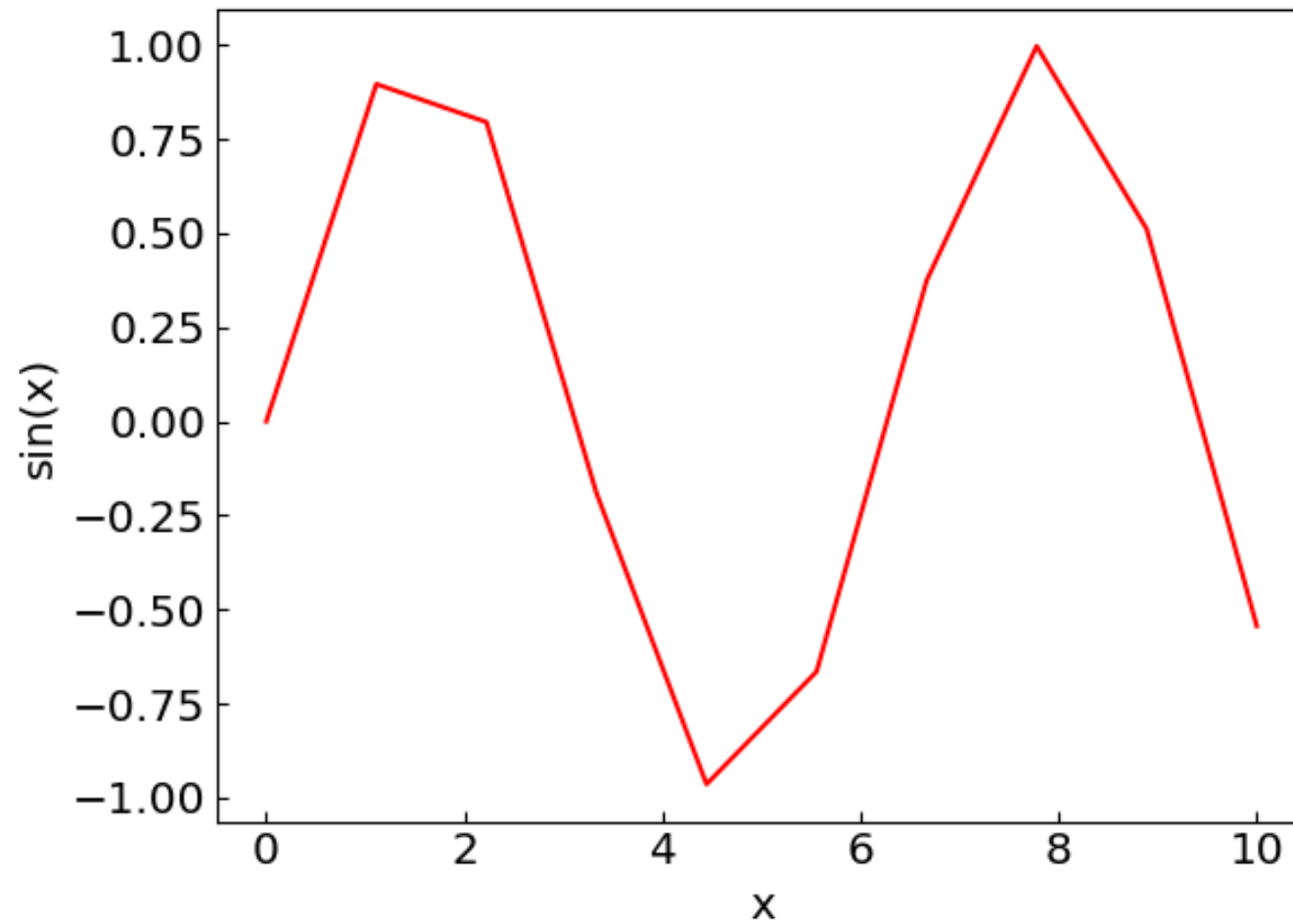
Now we have enough points to join them by a smooth line

Plot multiple lines to compare functions, profiles, etc.



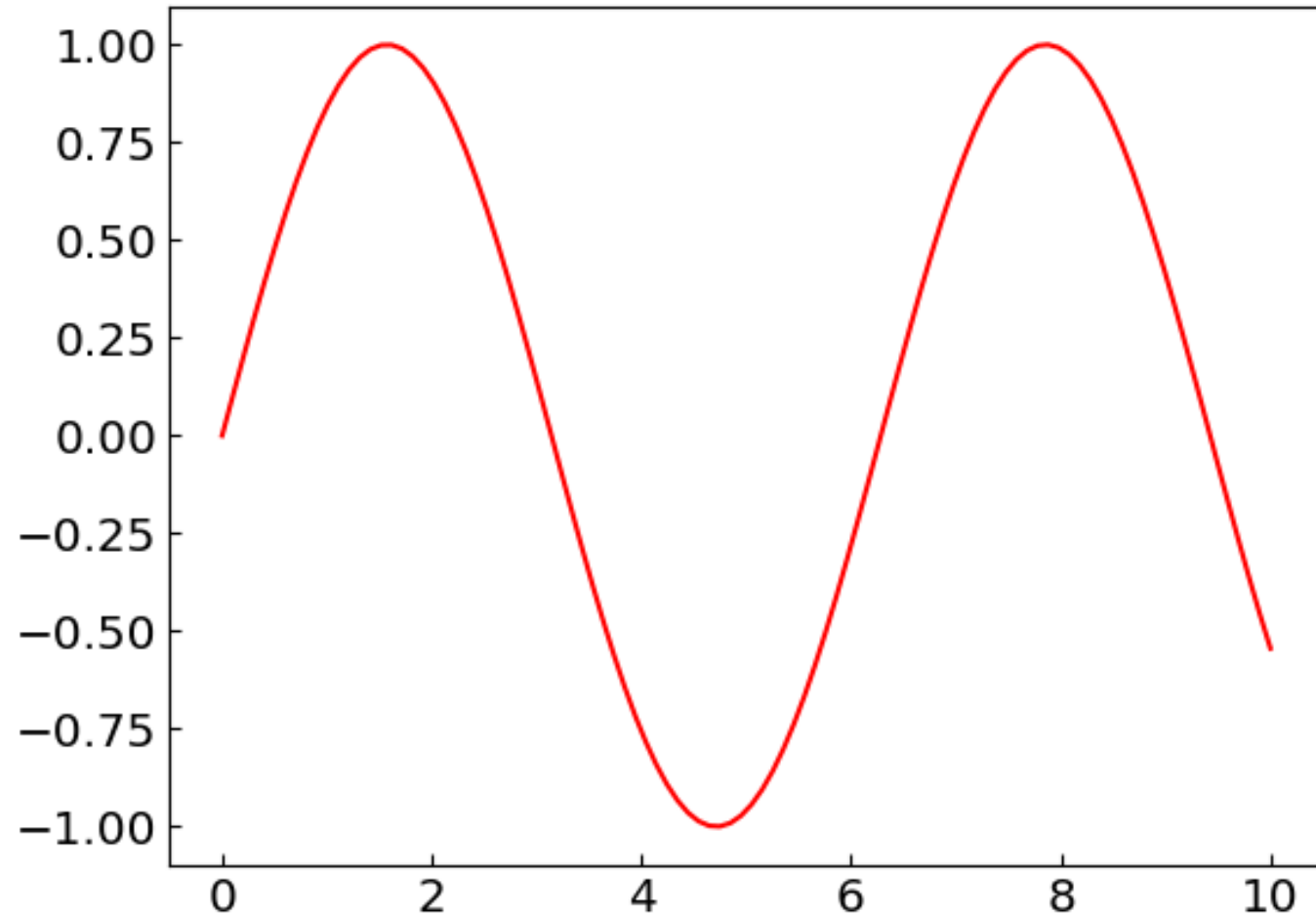
Things to avoid

Insufficient number of data points



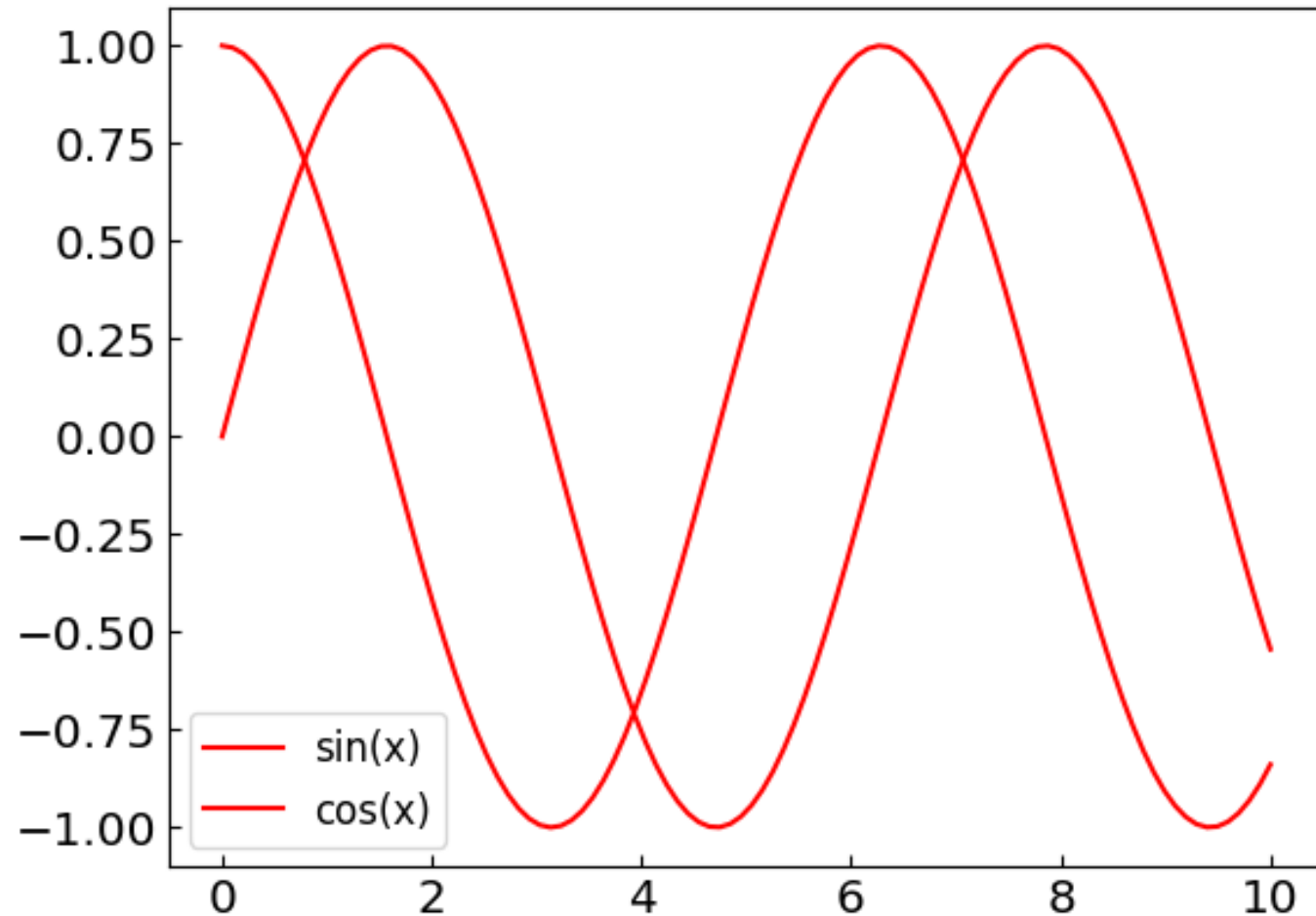
Things to avoid

Unlabeled axes



Things to avoid

Indistinguishable line styles

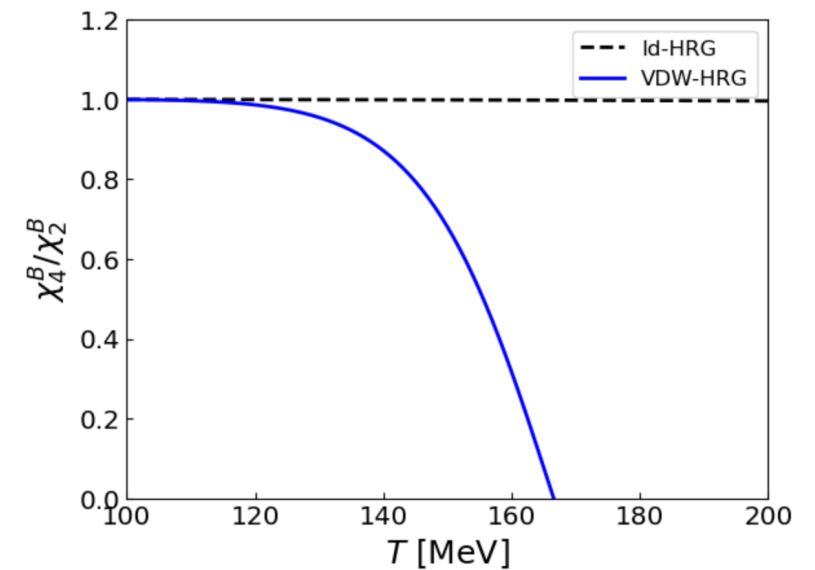
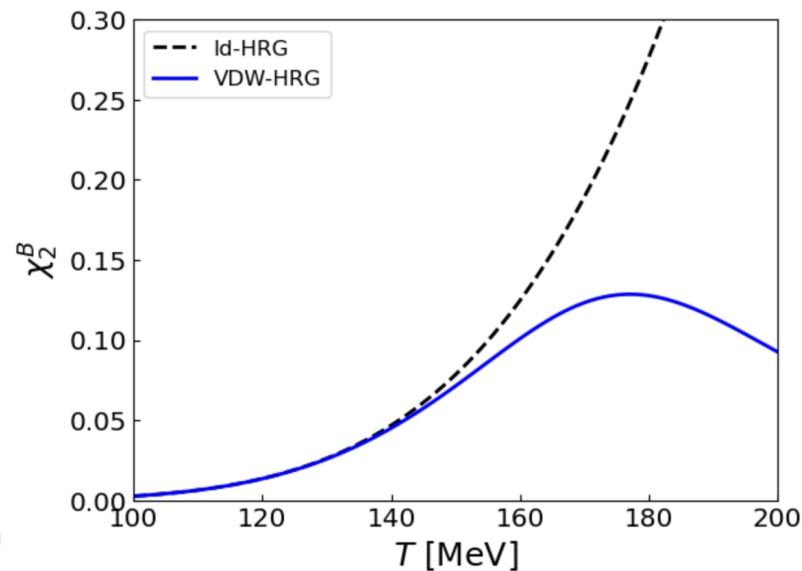
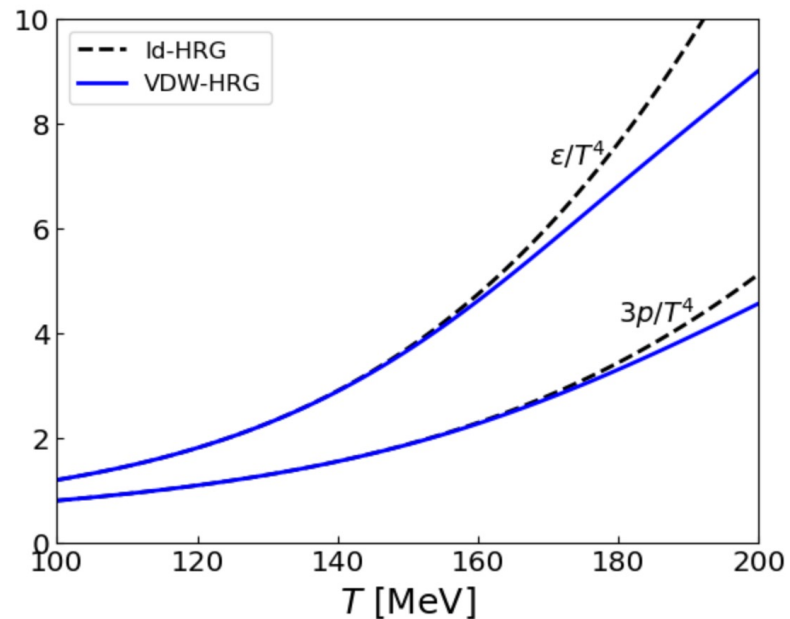


Plotting the HRG equation of state

Yesterday we computed the HRG equation of state with Thermal-FIST

Let us plot it (section “Plotting the HRG equation of state” in Plotting-NuSTEAM2024.ipynb)

You should obtain



Plotting the HRG equation of state

Published plots:

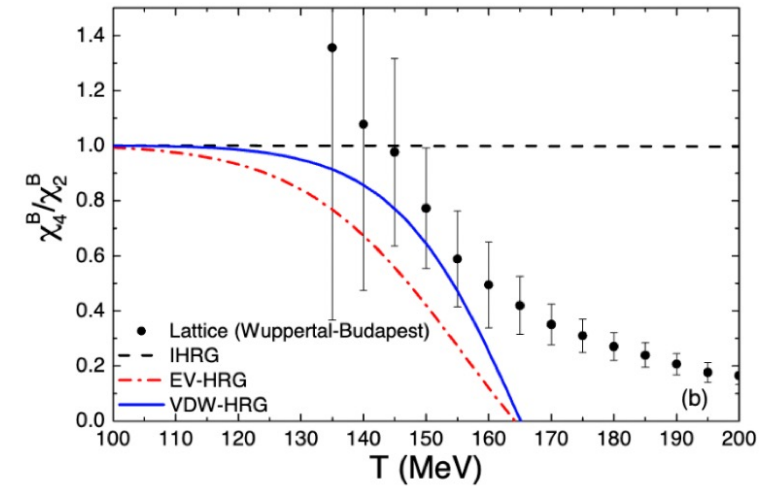
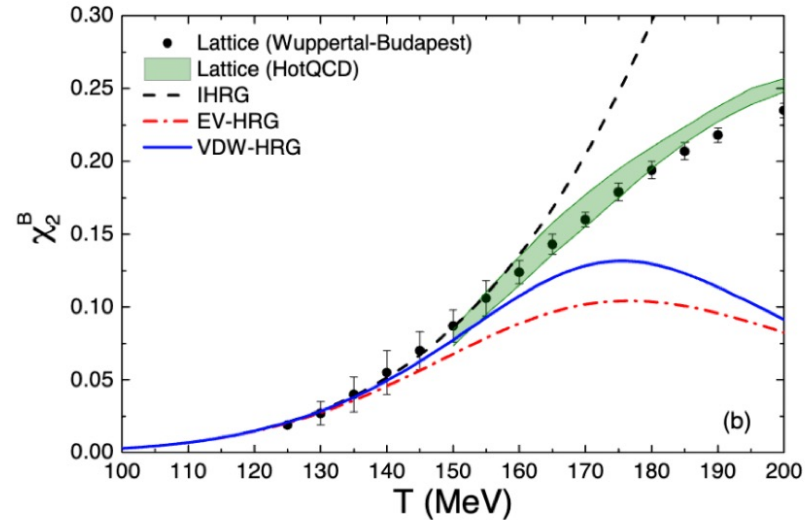
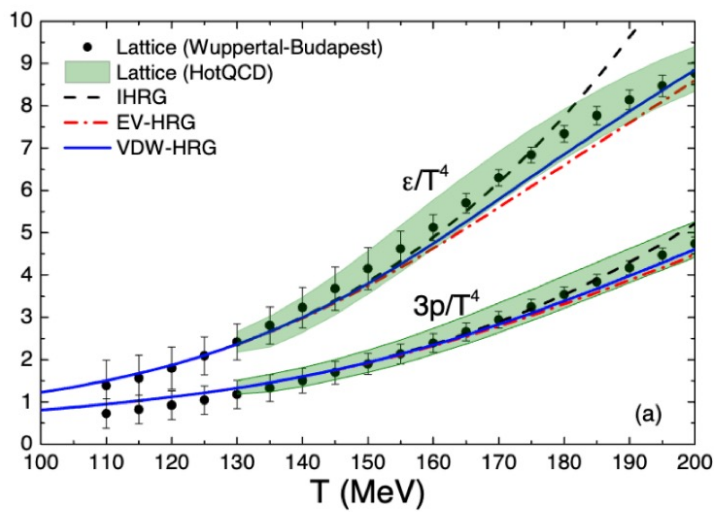


Figure from Vovchenko, Gorenstein, Stoecker, Phys. Rev. Lett. 118, 182301 (2017)

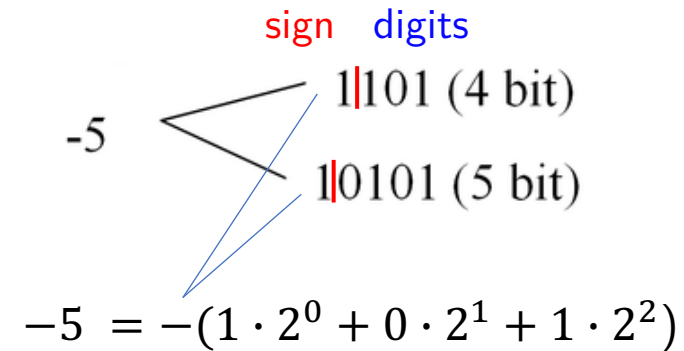
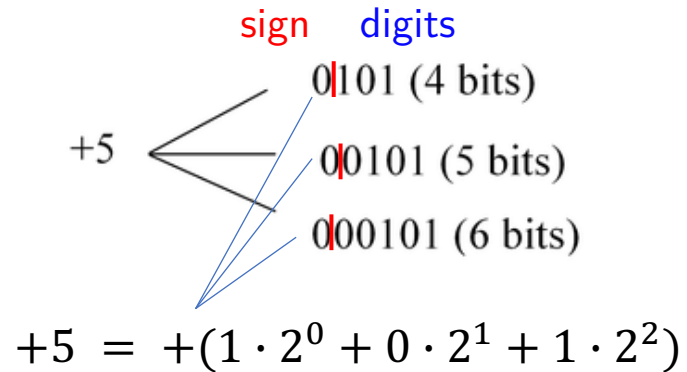
Exercise: Add EV-HRG curves by performing calculation for $a = 0$ and $b = 3.42 \text{ fm}^3$

Errors and accuracy

- Numbers representation
- Limits
- Floating-point precision
- Error accumulation

Integer representation

Numbers on a computer are represented by bits – sequences of 0s and 1s



Most typical native formats:

- 32-bit integer, range $-2,147,483,647$ (-2^{31}) to $+2,147,483,647$ (2^{31})
- 64-bit integer, range $\sim -10^{18}$ (-2^{63}) to $+10^{18}$ (2^{63})

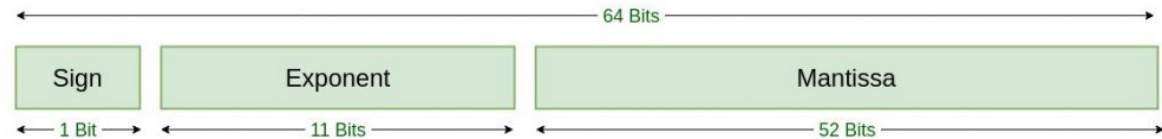
Python supports natively larger numbers but calculations can become slow

In C++ it is important to avoid under/over-flow

Floating-point number representation

Floating point numbers represented by bit sequences as well, which are separated into:

- Sign S
- Exponent E
- Mantissa M (significant digits)



Double Precision
IEEE 754 Floating-Point Standard

$$x = S \times M \times 2^{E-e}$$

e.g. $-2195.67 = -2.19567 \times 10^3$

Main consequence: Floating-point numbers are not exact!

For example, with 52 bits one can store about 16 decimal digits

Range: from $\sim -10^{308}$ to 10^{308} for a 64-bit float (double precision)

Floating-point number representation

What you write

$$x = 1.$$

What it means

$$x = 1. + \varepsilon_M, \quad \varepsilon_M \sim 10^{-16} \quad \text{for a 64-bit float}$$

Example: Equality test

```
x = 1.1 + 2.2

print("x = ",x)

if (x == 3.3):
    print("x == 3.3 is True")
else:
    print("x == 3.3 is False")
```

Example: Equality test

```
x = 1.1 + 2.2

print("x = ",x)

if (x == 3.3):
    print("x == 3.3 is True")
else:
    print("x == 3.3 is False")
```

```
x = 3.3000000000000003
x == 3.3 is False
```

Example: Equality test

```
x = 1.1 + 2.2

print("x = ",x)

if (x == 3.3):
    print("x == 3.3 is True")
else:
    print("x == 3.3 is False")
```

```
x = 3.3000000000000003
x == 3.3 is False
```

You can do instead

```
print("x = ",x)

# The desired precision
eps = 1.e-12

# The comparison
if (abs(x-3.3) < eps):
    print("x == 3.3 to a precision of",eps,"is True")
else:
    print("x == 3.3 to a precision of",eps,"is False")
```

```
x = 3.3000000000000003
x == 3.3 to a precision of 1e-12 is True
```

Error accumulation

$$x = 1. + \varepsilon_M, \quad \varepsilon_M \sim 10^{-16} \quad \text{unavoidable round-off error}$$

Errors accumulate through arithmetic operations, e.g.

$$y = \sum_{i=1}^N x_i$$

- $\sigma_y \sim \sqrt{N} \varepsilon_M$ if errors are independent
- $\sigma_y \sim N \varepsilon_M$ if errors are correlated
- σ_y can be large in some other cases

Two large numbers with a small difference

Let us have $x = 1$ and $y = 1 + \delta\sqrt{2}$

Symbolically, one has $\delta^{-1}(y - x) = \sqrt{2} = 1.41421356237\dots$

Let us test this relation on a computer for a very small value of $\delta = 10^{-14}$

```
from math import sqrt

delta = 1.e-14
x = 1.
y = 1. + delta * sqrt(2)
res = (1./delta)*(y-x)
print(delta, "* (y-x) = ", res)
print("The accurate value is sqrt(2) = ", sqrt(2))
print("The difference is ", res - sqrt(2))
```

```
1e-14 * (y-x) = 1.4210854715202004
The accurate value is sqrt(2) = 1.4142135623730951
The difference is 0.006871909147105226
```

Catastrophic loss of precision!

What happened?

	significant digits	these do not fit
y =	1.0000000000000014	142135623730951 ...
x =	1.0000000000000000	0000000000000000 ...

Other examples (hands-on session)

- Roots of a quadratic equation with $|ac| \ll b^2$
(cancellation of two large numbers with small difference)

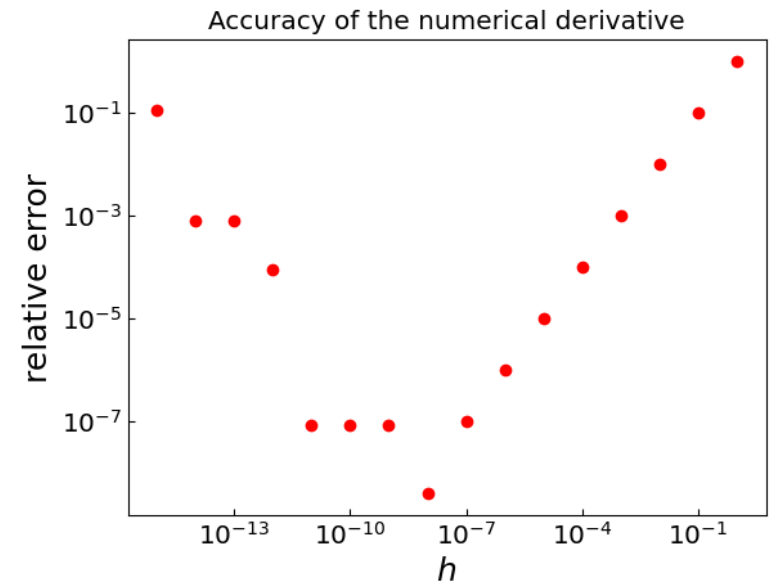
$$ax^2 + bx + c = 0$$

$$x = \frac{-b + \sqrt{b^2 - 4ac}}{2a}$$

- Simple numerical derivative

$$f'(x) \approx \frac{f(x+h) - f(x)}{h}$$

Sometimes a small h is too small



Quadratic equation

$$ax^2 + bx + c = 0$$

Symbolically, the roots are:

$$x_{1,2} = \frac{-b \pm \sqrt{b^2 - 4ac}}{2a}$$

$\sqrt{b^2 - 4ac}$ is very close to b

Let us calculate the roots for $a = 10^{-4}, b = 10^4, c = 10^{-4}$ $|ac| \ll b^2$

```
a = 1.e-4
b = 1.e4
c = 1.e-4

x1 = (-b + sqrt(b*b - 4.*a*c)) / (2.*a)
x2 = (-b - sqrt(b*b - 4.*a*c)) / (2.*a)

print("x1 = ", x1)
print("x2 = ", x2)
```

```
x1 = -9.094947017729282e-09
x2 = -100000000.0
```

x_2 looks ok *but x_1 seems off(?)*

Quadratic equation

$$ax^2 + bx + c = 0$$

Standard form:

$$x_{1,2} = \frac{-b \pm \sqrt{b^2 - 4ac}}{2a}$$

Alternative form:

$$x_{1,2} = \frac{2c}{-b \mp \sqrt{b^2 - 4ac}}$$

Using the alternative form

```
x1 = 2*c / (-b - sqrt(b*b-4.*a*c))
x2 = 2*c / (-b + sqrt(b*b-4.*a*c))

print("x1 = ", x1)
print("x2 = ", x2)
```

```
x1 = -1e-08
x2 = -109951162.7776
```

x_1 is fixed *but now x_2 is off*

Solution: Make a judicious choice between standard and alternative form for each root separately, such that subtraction of two similar number is avoided